| Practical: 6 | Solve a constraint satisfaction problem such as map coloring, Sudoku or N-Queens using backtracking and consistency checking. |
|---|---|

**AIM:**
To solve a Map Coloring Constraint Satisfaction Problem (CSP) using backtracking and consistency checking.

**THEORY:**
A Constraint Satisfaction Problem (CSP) consists of:
- Variables: objects that need values.
- Domains: possible values for each variable.
- Constraints: rules that restrict valid combinations.

For map coloring:
- Variables = regions/states
- Domain = colors
- Constraint = neighboring regions must have different colors.

Backtracking assigns a value to a variable and checks whether the assignment is consistent. If it causes a conflict, the algorithm goes back and tries another value.

**Algorithm**
1. Select an unassigned region.
2. Try a color from its domain.
3. Check whether the color conflicts with neighboring regions.
4. If there is no conflict, assign the color.
5. Move to the next region.
6. If no color is possible, backtrack to the previous region.
7. Continue until all regions are colored.

## PROGRAM:

```
map_data = {
   'A': ['B', 'C'],
   'B': ['A', 'C', 'D'],
   'C': ['A', 'B', 'D'],
   'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']
def is_consistent(region, color, assignment):
   for neighbor in map_data[region]:
      if neighbor in assignment and assignment[neighbor] == color:
         return False

   return True
def solve_map(assignment):
   # If every region has a color, we are done
   if len(assignment) == len(map_data):
      return assignment

   # Choose the next uncolored region
   for region in map_data:
      if region not in assignment:
         break

   # Try each color
   for color in colors:
      if is_consistent(region, color, assignment):
         assignment[region] = color

         result = solve_map(assignment)

         if result is not None:
            return result

         # Backtrack
         del assignment[region]

   return None
solution = solve_map({})

print("Solution:")

for region, color in solution.items():
   print(region, "=", color)
```

**OUTPUT:**

```
Solution:
A = Red
B = Green
C = Blue
D = Red
```

**CONCLUSION:**

In this practical, a Map Coloring CSP was solved using backtracking and consistency checking. The algorithm assigns colors to regions while checking constraints and backtracks whenever a conflict occurs. This demonstrates how CSP techniques can efficiently solve problems with multiple constraints.